02 - Arquitectura: Cómo Funciona Todo

Visión General de la Arquitectura

```
┌──────────────────────────────────────────────────────────────────┐
│                     Claude Code (MCP Client)                     │
│                   (Este IDE que estás usando)                    │
└──────────────────────────────┬───────────────────────────────────┘
                               │
                               │ MCP Protocol (JSON over stdio)
                               │
                  ┌────────────┴────────────┐
                  │                         │
     ┌────────────▼────────────┐  ┌─────────▼──────────────┐
     │   MCP Unity Server      │  │ MCP Blender Server     │
     │  (Node.js process)      │  │ (Node.js process)      │
     │                         │  │                        │
     │ - CallToolRequest       │  │ - CallToolRequest      │
     │ - ResourceRequest       │  │ - ResourceRequest      │
     │ - PromptRequest         │  │ - PromptRequest        │
     └────────────┬────────────┘  └────────────┬───────────┘
                  │                            │
                  │ WebSocket / CLI            │ CLI / Python
                  │                            │
     ┌────────────▼────────────┐  ┌────────────▼──────────┐
     │   Unity Editor          │  │   Blender             │
     │   (C# scripts)          │  │   (Python scripts)    │
     │   (Prefabs)             │  │   (Models)            │
     │   (GameObjects)         │  │   (Animations)        │
     │   (Assets)              │  │   (Exports)           │
     └────────────────────────┘  └──────────────────────┘
```

Componentes Principales

1. MCP Servers (Node.js)

Dos servidores independientes que corren como procesos Node.js:

MCP Unity Server

```
Escucha on: stdio (estándar input/output)
Comunica con: Claude Code (MCP protocol)
Ejecuta: Acciones en Unity via CLI o WebSocket
Herramientas (tools) que expone:
  - create_script
```

```
    - modify_script
    - delete_script
    - create_gameobject
    - configure_component
    - read_prefab
    - write_prefab
    - compile_project
    - get_compile_errors
    - build_game
```

MCP Blender Server

```
Escucha on: stdio
Comunica con: Claude Code (MCP protocol)
Ejecuta: Acciones en Blender via CLI y Python scripts
Herramientas (tools) que expone:
  - create_model
  - edit_model
  - delete_model
  - add_rig
  - create_animation
  - edit_animation
  - export_fbx
  - export_glb
  - batch_export
  - run_python_script
```

2. Unity Integration Layer

```
Unity Project
├── Assets/
├── ProjectSettings/
└── (rest of project)

Connection methods:
  a) CLI Mode (Recommended for solo dev)
     └─ Ejecutar Unity en batch mode desde CLI
        unity -batchmode -executeMethod ClassName.MethodName

  b) WebSocket Mode (For editor automation)
     └─ Script C# que escucha WebSocket
        Lee/escribe GameObjects en tiempo real
```

3. Blender Integration Layer

```
Blender Installation
└─ python3 blender_script.py
   └─ bpy (Blender Python API)
      ├─ Mesh operations
      ├─ Material creation
      ├─ Rigging (Armature)
      ├─ Animation
      └─ Export (FBX, GLB, etc.)
```

Flujo de Comunicación (Paso a Paso)

Ejemplo: Claude crea un script

```
1. Usuario escribes un prompt:
   "Crea un script PlayerController en Assets/Scripts/Player/"

2. Claude procesa el prompt
   └─ Identifica que es una acción "create_script"
   └─ Prepara los parámetros (path, content)

3. Claude llama a MCP Unity Server
   └─ Envía CallToolRequest:
      {
        "tool_name": "create_script",
        "arguments": {
          "path": "Assets/Scripts/Player/PlayerController.cs",
          "content": "... C# code ...",
          "namespace": "ZoremGame.Player"
        }
      }

4. MCP Unity Server recibe la request
   ├─ Valida los parámetros
   ├─ Escribe el archivo a disco
   ├─ Compila el proyecto (opcional)
   └─ Retorna resultado

5. Claude recibe la respuesta
   ├─ "Script creado exitosamente"
   ├─ "Compilación sin errores"
   └─ "Listo para ser usado"

6. Claude comunica a Claude Code
   └─ "Done! File created at Assets/Scripts/Player/PlayerController.cs"
```

Protocol MCP Detallado

Estructura de Mensaje

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "create_script",
    "arguments": {
      "path": "Assets/Scripts/Player/PlayerController.cs",
      "content": "public class PlayerController { ... }",
      "auto_compile": true
    }
  }
}
```

Respuesta Exitosa

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "Script created successfully at
Assets/Scripts/Player/PlayerController.cs"
      }
    ]
  }
}
```

Respuesta con Error

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "error": {
    "code": -32600,
    "message": "Path already exists",
    "data": {
      "details": "File Assets/Scripts/Player/PlayerController.cs already exists"
    }
  }
}
```

Herramientas (Tools) - Estructura

Tool Definition (Lo que claudia ve)

```
{
  "name": "create_script",
  "description": "Creates a new C# script in Unity",
  "inputSchema": {
    "type": "object",
    "properties": {
      "path": {
        "type": "string",
        "description": "Full path from Assets/ onwards (e.g.,
Scripts/Player/PlayerController.cs)"
      },
      "content": {
        "type": "string",
        "description": "C# source code"
      },
      "namespace": {
        "type": "string",
        "description": "C# namespace (e.g., ZoremGame.Player)",
        "default": "ZoremGame"
      },
      "auto_compile": {
        "type": "boolean",
        "description": "Auto-compile after creation",
        "default": false
      }
    },
    "required": ["path", "content"]
  }
}
```

Flujo Completo: De Prompt a Resultado

```
┌─ User escribe prompt
│  "Crea un Boss con IA básica"
│
├─ Claude análiza el prompt
│  "Esto requiere:
│   1. Modelo 3D en Blender
│   2. Rig para animación
│   3. Exportar a FBX
│   4. Prefab en Unity
│   5. Script de IA"
│
├─ Claude llama MCP Blender
│  ├─ create_model(specs)
```

```
│   ├─ add_rig(model)
│   └─ export_fbx(output_path)
│
├─ MCP Blender ejecuta
│   ├─ Genera modelo
│   ├─ Añade bones
│   ├─ Exporta FBX
│   └─ Retorna "Done"
│
├─ Claude llama MCP Unity
│   ├─ import_fbx(path)
│   ├─ create_prefab(gameobject)
│   ├─ create_script("BossAI.cs", ...)
│   └─ configure_component(animator)
│
├─ MCP Unity ejecuta
│   ├─ Importa el FBX
│   ├─ Crea el prefab
│   ├─ Genera el script
│   ├─ Compila
│   └─ Retorna "Done"
│
└─ Claude resume al usuario
    "Boss creado y listo. Ubicación: Assets/Prefabs/Bosses/Boss_01.prefab"
```

Configuración en Claude Code

settings.json

```
{
  "mcpServers": {
    "unity": {
      "command": "node",
      "args": ["/path/to/MCP/servers/unity/build/index.js"],
      "env": {
        "UNITY_PROJECT_PATH": "/Users/renat/Desktop/ZoremGame",
        "UNITY_EXECUTABLE":
"/Applications/Unity/Hub/Editors/2023.2.0f1/Unity.app/Contents/MacOS/Unity",
        "ASSET_STORE_PATH": "Assets",
        "AUTO_COMPILE": "true"
      }
    },
    "blender": {
      "command": "node",
      "args": ["/path/to/MCP/servers/blender/build/index.js"],
      "env": {
        "BLENDER_EXECUTABLE": "/Applications/Blender/blender",
        "BLENDER_PYTHON_VERSION": "3.10",
        "EXPORT_FORMAT": "fbx",
        "BATCH_MODE": "true"
```

```
      }
    }
  }
}
```

Estados y Ciclo de Vida

Estados del MCP Server

```
START
  ├─ Verificar que Unity/Blender esté instalado
  ├─ Validar rutas de proyecto
  ├─ Preparar handlers de herramientas
  └─ Escuchar en stdio

RUNNING
  ├─ Recibir CallToolRequest
  ├─ Procesar tool (ejecutar acciones)
  ├─ Retornar resultado
  └─ Mantenerse escuchando

SHUTDOWN
  ├─ Cerrar procesos hijos (Unity, Blender)
  ├─ Limpiar archivos temporales
  └─ Exit
```

Manejo de Errores

Estrategia de Error Handling

```
1. Validación de entrada
   └─ ¿Parámetros correctos?

2. Pre-ejecución
   └─ ¿Existen archivos/directorios requeridos?

3. Ejecución
   └─ ¿El proceso Unity/Blender se ejecutó correctamente?

4. Post-ejecución
   └─ ¿Compilación sin errores?
   └─ ¿Archivo generado correctamente?

5. Fallback
   └─ Si falla, proporcionar mensaje claro
   └─ Sugerir acciones de recover
```

Ejemplo de Error

```
Tool: create_script
Path: "Assets/Scripts/Player/PlayerController.cs"

Validación: ❌ File already exists
Response:
{
  "error": "File exists",
  "message": "Assets/Scripts/Player/PlayerController.cs already exists",
  "suggestions": [
    "1. Delete the existing file",
    "2. Create with different name (e.g., PlayerController_v2.cs)",
    "3. Overwrite with force flag: create_script(..., force=true)"
  ]
}
```

Seguridad y Sandboxing

Limitaciones Intencionales

```
1. Rutas permitidas
   └─ Solo dentro de Project Folder (Assets/)
   └─ No permite "../../../" path traversal

2. Comandos permitidos
   └─ Solo herramientas explícitamente registradas
   └─ No permite ejecutar comandos arbitrarios

3. Permisos de archivos
   └─ Solo lectura/escritura en carpeta Assets/
   └─ No puede borrar ProjectSettings/

4. Limits de recursos
   └─ Timeout en herramientas (30 sec default)
   └─ Límite de file size (100MB default)
```

Performance y Optimización

Consideraciones

```
1. Cold start
   └─ Primer llamada: lenta (Unity/Blender tarda en iniciar)
   └─ Solución: Mantener proceso activo en background
```

```
2. Síncronización
   └─ Evitar llamadas simultáneas a mismo recurso
   └─ Queue de operaciones

3. Batch operations
   └─ En lugar de N llamadas separadas
   └─ Una llamada que procesa N items

4. Caching
   └─ Cache de rutas de archivos
   └─ Cache de configuración de proyecto
```

Diagrama de Dependencias

```
Claude Code
    ↓
┌──────────────────────────────────────┐
│   MCP Protocol (stdio + JSON-RPC)    │
└──────────────────────────────────────┘
    ↓                          ↓
┌──────────────────────┐  ┌──────────────────────┐
│  MCP Unity Server    │  │ MCP Blender Server   │
├──────────────────────┤  ├──────────────────────┤
│  - CLI handler       │  │  - CLI handler       │
│  - File I/O          │  │  - File I/O          │
│  - Compilation       │  │  - Process manager   │
│  - Asset management  │  │  - Python runner     │
└──────────────────────┘  └──────────────────────┘
    ↓                          ↓
┌──────────────────────┐  ┌──────────────────────┐
│  Unity Editor        │  │   Blender            │
│  - C# compilation    │  │   - 3D modeling      │
│  - Asset handling    │  │   - Animation        │
│  - Prefabs           │  │   - Export (FBX)     │
└──────────────────────┘  └──────────────────────┘
```

Siguiente: Implementación

Ahora que entiendes la arquitectura, sigamos a:

- **03_Unity_MCP.md** - Cómo construir el MCP Unity paso a paso